

Actividad Práctica

Uso de las Bases de Datos NoSQL

Autor:
MARLON CÁRDENAS BONETT

Actualizado diciembre 2025

INTRODUCCION

En un mundo donde los datos se han convertido en un recurso muy valioso, las bases de datos NoSQL han emergido como una solución clave en la arquitectura de datos moderna. Más que una alternativa a los sistemas relacionales tradicionales, estas bases de datos representan un cambio fundamental en la forma en que almacenamos, gestionamos y analizamos información.

Vivimos en una era de datos masivos, caracterizada por la velocidad, el volumen y la variedad de información generada por fuentes tan diversas como redes sociales, sensores IoT y aplicaciones en tiempo real. Estas características han superado las capacidades tradicionales de las bases de datos relacionales, y es aquí donde las bases de datos NoSQL destacan, ofreciendo soluciones que las hacen indispensables en entornos exigentes:

- Manejo de grandes volúmenes de datos: gracias a su capacidad de escalar horizontalmente, las bases de datos NoSQL permiten manejar cantidades masivas de información, desde terabytes hasta petabytes, de manera eficiente y rentable, algo que sería inviable en sistemas relacionales.
- Flexibilidad en la estructura de datos: diseñadas para adaptarse a estructuras no rígidas, las bases de datos NoSQL son ideales para proyectos dinámicos y negocios en constante evolución. Su capacidad para manejar datos semiestructurados o sin estructura las convierte en una herramienta perfecta para experimentación y crecimiento.
- Rendimiento optimizado y velocidad: al estar diseñadas para consultas específicas, estas bases de datos ofrecen un rendimiento excepcionalmente rápido, una característica crucial en aplicaciones que requieren respuestas en tiempo real, como sistemas de recomendación, análisis en *streaming* o plataformas de comercio electrónico.

OBJETIVO DE LA ACTIVIDAD

El principal objetivo de esta práctica es utilizar datos de diferentes fuentes para diseñar y proponer un modelo de datos efectivo **sobre sistemas NoSQL**. Esto nos permitirá comprender cómo la estructura y el diseño de datos pueden facilitar significativamente la consulta y manipulación de la información en entornos no relacionales.

ENFOQUE EN MONGODB Y NEO4J:

- **MongoDB:** Una base de datos orientada a documentos, que nos permite trabajar con datos en un formato que se asemeja a JSON. Esto es ideal para manejar grandes volúmenes de datos con una estructura variada, proporcionando flexibilidad y una fácil escalabilidad.
- **Neo4J:** Un sistema de base de datos orientado a grafos, perfecto para representar relaciones complejas entre datos. Con Neo4J, podemos explorar conexiones y patrones que serían difíciles de visualizar y manejar en sistemas de bases de datos tradicionales.

ESTRUCTURA DE LA ACTIVIDAD:

- **Introducción Teórica:** Breve repaso sobre los conceptos clave de las bases de datos NoSQL, con énfasis en MongoDB y Neo4J.
- **Análisis de Datos:** Selección y análisis de un conjunto de datos para identificar requisitos y desafíos específicos.
- **Diseño de Modelos de Datos:** Creación de esquemas y modelos de datos utilizando MongoDB y Neo4J, enfocándose en optimizar la consulta y manipulación de datos.
- **Implementación Práctica:** Configuración de entornos de bases de datos y realización de operaciones CRUD (Crear, Leer, Actualizar, Eliminar) en los modelos diseñados.
- **Análisis y Comparación:** Evaluación del rendimiento, flexibilidad y eficiencia de cada modelo en distintos escenarios de consulta de datos.

RUBRICA

		NOTA	PESO
RETO 1	Realizar la carga de los datos elegidos y llevar a cabo una exploración de estos utilizando el motor de base de datos propuesto.	2.0	20.0%
RETO 2	Construir un Modelo de datos siguiendo uno de los patrones conocidos. El modelo debe ser flexible y responder a todas las consultas propuestas.	4.0	40.0%
RETO 3	Construir un modelo de datos orientado a grafos con el que sea posible cubrir las consultas propuestas. Representar nodos, relaciones y atributos utilizando los datos disponibles y el modelo de datos del apartado anterior.	4.0	40.0%
NOTA FINAL:		10.0	100.0%

ENTREGA

- Elabora un **informe** con los elementos habituales (portada, índice, nombre, fecha, numeración de páginas, etc.), en el que des respuesta a los retos planteados.
- Cada reto se debe responder en una sección diferente. El informe debe incluir la respuesta de cada literal, si es necesario incluir consultas (script), capturas de pantalla, explicaciones, etc.

RETO 1 -EXPLORACIÓN INICIAL DE LOS DATOS

1. **Revisión inicial:** examina los ficheros proporcionados y la documentación asociada a cada uno de ellos. Asegúrate de comprender el contexto de los datos y sus características principales antes de comenzar con la exploración. Comprueba que la estructura y el formato sean adecuados para su manipulación.
 - a. Utiliza herramientas como Python (pandas) para explorar y analizar la calidad de los datos antes de su carga en MongoDB.
2. **Revisión de campos:** analiza los campos de cada fichero, identificando su contenido, tipos de datos y posibles inconsistencias. Evalúa la calidad de los datos, buscando errores comunes como valores nulos, duplicados o inconsistencias en los formatos.
 - a. No tienes que realizar una revisión profunda y tampoco limpiar los datos en su totalidad, basta con comprobar algunos documentos de muestra.
3. **Relaciones entre ficheros:** identifica, siempre que sea posible, un campo o conjunto de campos que permita establecer relaciones entre los datos de los distintos ficheros. Considera que, al trabajar con datos no estructurados, estas relaciones podrían no ser evidentes o factibles.
 - a. Si no es posible establecer una relación clara, analiza el contexto y propón una alternativa para manejar los datos, como almacenarlos de forma separada o diseñar estrategias para su integración futura.

Nota: la relación entre los datos no siempre será viable debido a las características de los ficheros o a la falta de correspondencia directa entre ellos. En estos casos, justifica tu decisión y describe cómo planeas gestionar los datos en MongoDB.

RETO 2 - MODELADO DE DATOS

Después de haber explorado y comprendido la estructura de los ficheros, realiza las siguientes actividades.

1. Diseño del modelo de datos (Versión 1)

- 1.1. **Propuesta de modelo de datos:** diseña un modelo de datos basado en documentos para almacenar la información de los ficheros de manera eficiente y accesible. Puedes optar por un enfoque **embebido**, **referenciado** o **jerárquico**, según los patrones vistos en este curso y la naturaleza de los datos.

1.2. Implementación del modelo: crea una base de datos en MongoDB e implementa el modelo propuesto, cargando los ficheros en las colecciones correspondientes.

2. Uso del modelo mediante consultas

2.1 Diseña y ejecuta las siguientes consultas en MongoDB para evaluar la eficiencia y funcionalidad de tu modelo de datos. Cada consulta incluye una descripción del ejercicio junto con una pista para ayudarte a construirla.

- a. El Total de locales y terrazas por distrito y barrio: construye una consulta que permita obtener el total de locales y terrazas agrupados por cada distrito y barrio.
 - **Pista:** *identifica en tu modelo los campos correspondientes a distrito y barrio. Usa un pipeline de agregación y aplica \$group para agrupar los documentos por estas categorías. Utiliza \$sum para contar el total de locales y terrazas en cada grupo.*
- b. Tipos de licencias y cantidad de licencias por cada tipo: crea una consulta para contar cuántas licencias hay de cada tipo en los datos.
 - **Pista:** *encuentra el campo que indica el tipo de licencia. Utiliza \$group para agrupar por este campo y \$count o \$sum para calcular el número de licencias en cada grupo. Si encuentras valores nulos o inconsistentes, considera cómo manejarlos.*
- c. Listado de locales y terrazas con licencias “En trámite”: diseña una consulta que filtre y devuelva un listado detallado de locales y terrazas cuyo estado de licencia sea “En trámite”.
 - **Pista:** *localiza el campo que representa el estado de la licencia. Usa el operador \$match para filtrar los documentos donde este campo sea igual a “En trámite”. Piensa también en cómo manejar mayúsculas, minúsculas u otros formatos en el texto.*
- d. Consulta por sección, división y epígrafe de la actividad comercial: crea una consulta para clasificar locales y terrazas según los campos sección, división y epígrafe.
 - **Pista:** *busca estos tres campos en tu modelo y utiliza \$and o construye un filtro que combine las condiciones necesarias para devolver los resultados clasificados. Considera cómo manejar casos donde uno de estos campos pueda estar vacío.*

- e. Actividad económica más frecuente por barrio y distrito: diseña una consulta que identifique cuál es la actividad económica predominante en cada barrio y distrito.
- **Pista:** *agrupa los datos por barrio y distrito usando \$group. Luego, dentro de cada grupo, cuenta la frecuencia de cada actividad económica (campo correspondiente) y utiliza \$sort o \$max para encontrar la más frecuente. Piensa en cómo combinar varias etapas del pipeline para lograr esto.*
- f. Actualización de horarios de apertura y cierre de ciertos locales: modifica los horarios de apertura y cierre de un conjunto seleccionado de locales según un criterio que tú determines.
- **Pista:** *primero, define el criterio que utilizarás para seleccionar los locales a modificar (por ejemplo, su ubicación o actividad económica). Usa \$updateMany o \$updateOne en combinación con \$set para realizar los cambios. Documenta el criterio elegido en tu informe y explica por qué es relevante.*

3. Creación y uso de índices en MongoDB

En este ejercicio, trabajarás con la creación de diferentes tipos de índices (simples, compuestos y de array) para optimizar la búsqueda y manipulación de datos en MongoDB.

- a. **Índice simple:** crea un índice sobre un campo único, como el nombre del barrio. Asegúrate de que la búsqueda de documentos en base a este campo sea más eficiente.
- **Pista:** Utiliza el comando `db.collection.createIndex({ campo: 1 })` para crear un índice en orden ascendente. Luego, ejecuta una consulta que filtre los datos por barrio para observar el impacto del índice en el rendimiento.
- b. **Índice compuesto:** diseña un índice compuesto que combine los campos distrito y barrio. Este índice debe mejorar el rendimiento en consultas que incluyan ambos campos.
- **Pista:** usa el comando `db.collection.createIndex({ campo1: 1, campo2: 1 })`. Recuerda que el orden de los campos en un índice compuesto es importante, así que considera cómo suelen ser tus consultas para decidir el orden adecuado.
- c. **Índice de array:** si tienes un campo que almacena un array (por ejemplo, una lista de actividades económicas asociadas a un local o terraza), crea un índice sobre ese campo para permitir búsquedas rápidas de documentos que contengan valores específicos en el array.

- **Pista:** usa el comando `db.collection.createIndex({ campo: 1 })`. Luego, realiza una consulta para buscar documentos que incluyan una actividad económica específica, como `{ actividades_economicas: "Restauración" }`.

Investiga: después de crear los índices, utiliza el comando `explain()` para comparar el plan de ejecución de las consultas con y sin índices. Describe en tu informe cómo estos índices han mejorado (o no) el rendimiento en cada caso.

4. Modelo de datos (Versión 2): Extensión con alojamientos turísticos

Con base en el modelo de datos actual, que contiene información sobre la actividad económica de locales y terrazas en la ciudad de Madrid, amplía y mejora el modelo para incluir datos relacionados con alojamientos turísticos de los distintos barrios de la ciudad. Realiza las siguientes tareas:

4.1. Incorporación de datos de alojamientos turísticos

- a. **Revisión y obtención del dataset:** accede a la URL proporcionada (<http://insideairbnb.com/new-york-city/>) y localiza el dataset correspondiente a los alojamientos de Airbnb en Madrid. Asegúrate de descargar los datos en el formato adecuado (CSV o JSON) para su posterior uso.
 - **Pista:** examina la estructura del dataset descargado y compárala con los datos ya cargados en tu base de datos para identificar similitudes y diferencias. Si encuentras datos innecesarios o irrelevantes, realiza una limpieza previa antes de cargar el dataset en MongoDB.
- b. **Carga de datos en MongoDB:** importa el dataset a la misma base de datos donde alojas tu modelo actual. Decide si los datos de alojamientos estarán en una colección independiente o si serán embebidos/referenciados en el modelo existente.
 - **Pista:** considera la relación entre alojamientos y barrios para decidir cómo integrarlos en el modelo. Usa patrones de datos embebidos o referenciados según sea necesario. Si optas por una colección independiente, establece una relación clara con los barrios o distritos para facilitar las consultas posteriores.

4.2. Mejora del modelo de datos

- a. **Propuesta de mejora del modelo:** diseña modificaciones o ampliaciones en tu modelo de datos actual para incorporar la información de alojamientos turísticos. Justifica las decisiones tomadas, asegurándote de mantener un diseño eficiente para las consultas posteriores.

- **Pista:** piensa en cómo relacionar los alojamientos con la actividad comercial (locales y terrazas) usando un campo común, como barrio o distrito. Si el dataset contiene información sobre servicios o características de los alojamientos, evalúa cómo representarlas en el modelo (por ejemplo, como un array o subdocumentos).

4.3. Consultas para validar el modelo

Diseña y ejecuta las siguientes consultas para demostrar la eficiencia del modelo mejorado:

a. Total de alojamientos, locales y terrazas por distrito y barrio.

- **Pista:** usa un pipeline de agregación para agrupar los datos por distrito y barrio, sumando los totales de cada tipo (alojamientos, locales y terrazas) en un solo resultado.

b. Los barrios con mayor número de alojamientos y terrazas con licencias concedidas en los últimos dos años.

- **Pista:** filtra los datos de terrazas por fecha de concesión de licencia (asegúrate de tener este campo en un formato procesable, como ISODate). Luego, agrupa por barrio y selecciona los que tengan los mayores conteos de alojamientos y terrazas combinados.

RETO 3 - VISUALIZA LOS DATOS: MODELO DE GRAFO

1. **Diseño del modelo de grafo:** para este ejercicio, no es necesario utilizar o cargar todos los datos del dataset anterior.

- Se sugiere trabajar con un subconjunto representativo de los datos o crear ejemplos que reflejen las estructuras y relaciones principales.
- El objetivo es demostrar la utilidad del grafo y cómo puede facilitar las consultas y análisis para los datos seleccionados.
- Asegúrate de que los ejemplos sean suficientes para evidenciar los beneficios del enfoque basado en grafos.
- El diseño debe ser visual, representa gráficamente tu modelo de grafo.

1.1. **Nodos y sus atributos:** define los nodos del grafo para representar las entidades principales.

Pista: Nodos sugeridos:

- *Local:* atributos como nombre, tipo de actividad, horario, licencia.
- *Terraza:* atributos como nombre, capacidad, estado de licencia.
- *Alojamiento:* atributos como nombre, precio, número de habitaciones, reseñas.
- *Barrio:* atributo principal nombre.

1.2. **Etiquetas de nodos:** propón etiquetas que ayuden a diferenciar los nodos y permitan consultas específicas en el grafo.

Pista: Etiquetas sugeridas: *Local, Terraza, Alojamiento, Barrio*

1.3. **Relaciones y sus atributos:** define las conexiones entre los nodos y los atributos que estas relaciones deben incluir.

Pista: Relaciones sugeridas:

- *Ubicado_en:* conecta Local, Terraza y Alojamiento con su Barrio. Atributo: distancia.
- *Cercano_a:* conecta Local, Terraza y Alojamiento entre sí si están a menos de un umbral de distancia definido (por ejemplo, 500 metros). Atributo: distancia.
- *Relacionado_con:* conecta nodos de la misma categoría (Local, Terraza, o Alojamiento) que comparten atributos comunes, como tipo de actividad o precio.

2. Consultas sobre el grafo

2.1. Consulta todos los locales y terrazas del barrio “Salamanca”.

Pista: Busca nodos etiquetados como Local y Terraza que estén conectados al nodo del barrio Salamanca mediante la relación Ubicado_en.

```
MATCH (b:Barrio {nombre: "Salamanca"})<-[:Ubicado_en]-(n)
WHERE n:Local OR n:Terraza
RETURN n.nombre AS nombre, labels(n) AS tipo
```

2.2. Consulta los alojamientos cuyo precio supere los 100€.

Pista: Filtra los nodos etiquetados como Alojamiento cuyos atributos incluyan precio > 100.

```
MATCH (a:Alojamiento)
WHERE a.precio > 100
RETURN a.nombre AS nombre, a.precio AS precio
```

2.3. Consulta los barrios donde los alojamientos no cuentan con dormitorios.

Pista: Busca nodos etiquetados como Alojamiento donde el atributo número de habitaciones sea igual a 0 y obtén los barrios conectados mediante la relación Ubicado_en.

```
MATCH (a:Alojamiento)-[:Ubicado_en]->(b:Barrio)
WHERE a.numero_habitaciones = 0
RETURN b.nombre AS barrio, COUNT(a) AS total_alojamientos
```

2.4. Consulta los barrios donde los alojamientos no cuentan con reseñas.

Pista: Filtra nodos etiquetados como Alojamiento donde el atributo reseñas sea igual a 0 y devuelve los barrios relacionados mediante la conexión Ubicado_en.

```
MATCH (a:Alojamiento)-[:Ubicado_en]->(b:Barrio)
WHERE a.reseñas = 0
RETURN b.nombre AS barrio, COUNT(a) AS total_alojamientos
```

PISTA: Apóyate en las siguientes instrucciones para crear el grafo.

Crear nodos y sus atributos

Crear nodos de barrios:

```
CREATE (b1:Barrio {nombre: "Salamanca"})
CREATE (b2:Barrio {nombre: "Chamberí"})
CREATE (b3:Barrio {nombre: "Centro"})
```

Los nodos Barrio se crean con el atributo nombre, que identifica cada barrio.

Crear nodos de locales:

```
CREATE (l1:Local {nombre: "Cafetería Goya", tipo_actividad: "Restauración",  
horario: "08:00-22:00", licencia: "Concedida"})  
CREATE (l2:Local {nombre: "Librería Central", tipo_actividad: "Cultura",  
horario: "10:00-20:00", licencia: "En trámite"})
```

Los nodos Local incluyen atributos como nombre, tipo_actividad, horario y licencia.

Crear nodos de terrazas:

```
CREATE (t1:Terraza {nombre: "Terraza Sol", capacidad: 30, estado_licencia:  
"Concedida"})  
CREATE (t2:Terraza {nombre: "Terraza Norte", capacidad: 20, estado_licencia:  
"En trámite"})
```

Los nodos Terraza tienen atributos como nombre, capacidad y estado_licencia.

Crear nodos de alojamientos:

```
CREATE (a1:Alojamiento {nombre: "Airbnb Retiro", precio: 120,  
numero_habitaciones: 2, reseñas: 45, servicios: ["WiFi", "Cocina"]})  
CREATE (a2:Alojamiento {nombre: "Apartamento Goya", precio: 80,  
numero_habitaciones: 1, reseñas: 0, servicios: ["Aire acondicionado"]})
```

Los nodos Alojamiento incluyen atributos como precio, numero_habitaciones, reseñas y servicios, donde los servicios están representados como un array.

Crear relaciones y sus atributos: relacionar locales, terrazas y alojamientos con barrios:

```
MATCH (b:Barrio {nombre: "Salamanca"}), (l:Local {nombre: "Cafetería Goya"})  
CREATE (l)-[:Ubicado_en {distancia: 0.5}]->(b)
```

```
MATCH (b:Barrio {nombre: "Centro"}), (a:Alojamiento {nombre: "Airbnb  
Retiro"})  
CREATE (a)-[:Ubicado_en {distancia: 0.3}]->(b)
```

La relación *Ubicado_en* conecta locales, terrazas y alojamientos con barrios, e incluye un atributo distancia que indica la proximidad al centro del barrio.

Relacionar entidades cercanas entre sí:

```
MATCH (l:Local {nombre: "Cafetería Goya"}), (t:Terraza {nombre: "Terraza  
Sol"})  
CREATE (l)-[:Cercano_a {distancia: 0.2}]->(t)
```

```
MATCH (a:Alojamiento {nombre: "Airbnb Retiro"}), (l:Local {nombre: "Librería  
Central"})  
CREATE (a)-[:Cercano_a {distancia: 0.1}]->(l)
```

La relación *Cercano_a* conecta entidades como locales, terrazas y alojamientos que están físicamente próximas. El atributo distancia mide la cercanía en kilómetros.

Relacionar nodos con atributos comunes:

```
MATCH (l:Local {tipo_actividad: "Restauración"}), (t:Terraza {nombre:  
"Terraza Sol"})  
CREATE (l)-[:Relacionado_con {motivo: "Ambiente de comida"}]->(t)
```

La relación *Relacionado_con* conecta nodos de diferentes tipos que comparten características o contextos similares. El atributo motivo explica la relación.

Consultas para verificar la creación del grafo: visualizar todos los nodos y relaciones:

```
MATCH (n)-[r]->(m)
RETURN n, r, m
```

Verificar los nodos y atributos creados:

```
MATCH (n:Local)
RETURN n.nombre, n.tipo_actividad, n.horario, n.licencia
```

Consultar relaciones y sus atributos:

```
MATCH (n)-[r:Ubicado_en]->(m:Barrio)
RETURN n.nombre AS entidad, m.nombre AS barrio, r.distancia AS distancia
```

IMPORTANTE: en el Reto 3, las pitas te ofrecen toda la información que necesitas para crear tu modelo de grafo y realizar las consultas. Puedes proponer nodos, relaciones y atributos nuevos, lo cual es ideal, pero también puedes usar las pistas directamente para desarrollar este apartado. Esto quiere decir que el reto 3 es básicamente comprender el grafo sugerido (punto 1) y seguir las pistas para comprobar su utilidad (punto 2).

Entrega final:

Presenta un informe **en PDF** que incluya:

1. La descripción del dataset incorporado y los pasos realizados para su limpieza y carga.
2. El modelo de datos actualizado, explicando las mejoras realizadas.
3. Los resultados de las consultas junto con el análisis de su eficiencia y cómo reflejan las mejoras del modelo.
4. Documenta los pasos anteriores e incluye las capturas oportunas para justificar cada paso desarrollado.
5. **Las consultas construidas deben estar en texto**, no en capturas. El resultado de las consultas sí puede estar como captura, y no es necesario mostrar la salida completa, basta con una pequeña imagen de ello.
6. No es necesario subir los dataset generados, basta una muestra de ellos (5 documentos, 5 registros, etc.).
7. El notebook se puede adjuntar **solo en formato PDF** (exportar a PDF).
8. **No se esperan script de Python ni nada parecido.**